sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

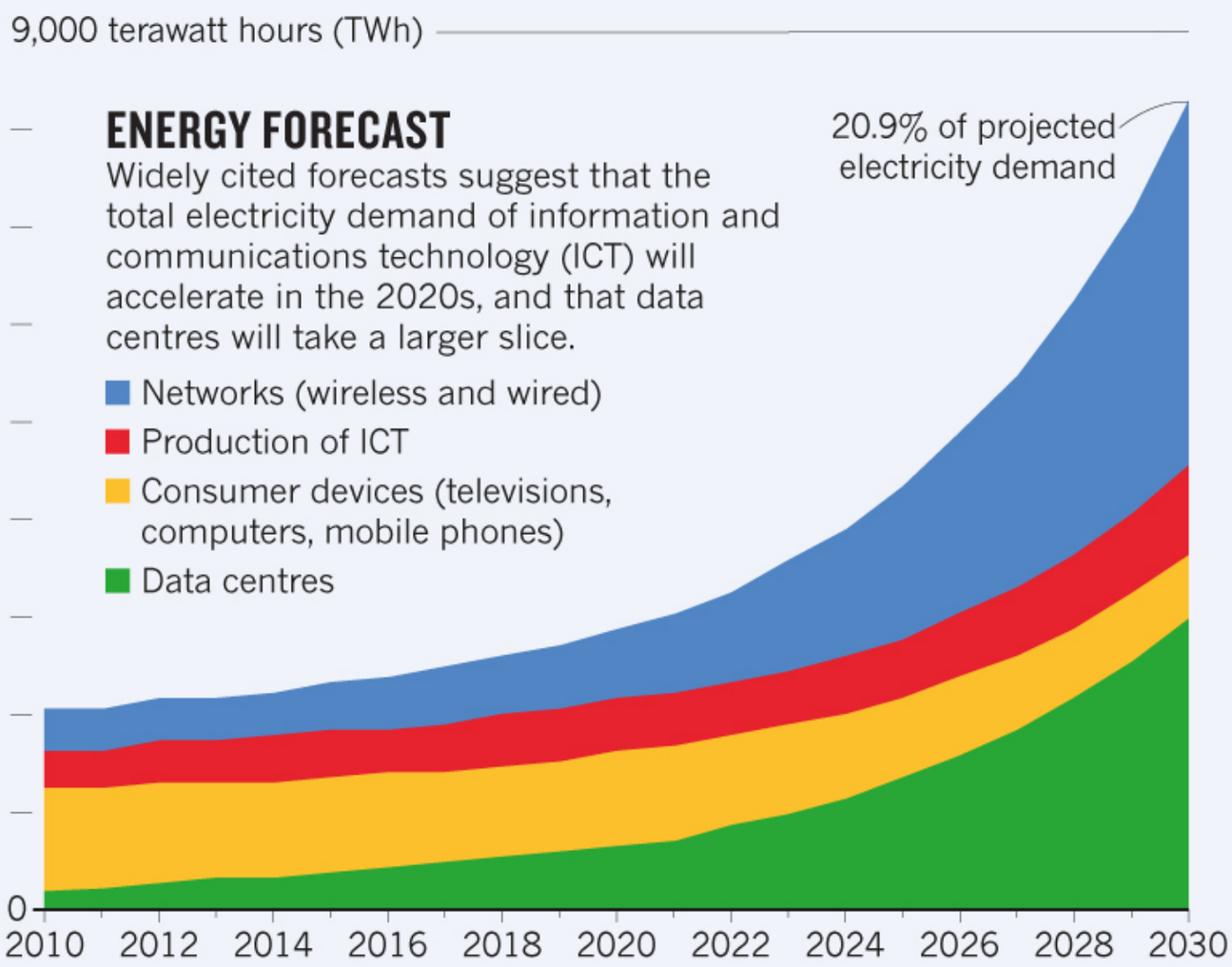
Sustainable Software Engineering - Criteria for Sustainable Software Products



Gemma Catolino | 2026-2027

Sustainable Software Engineering: ICT and Sustainability

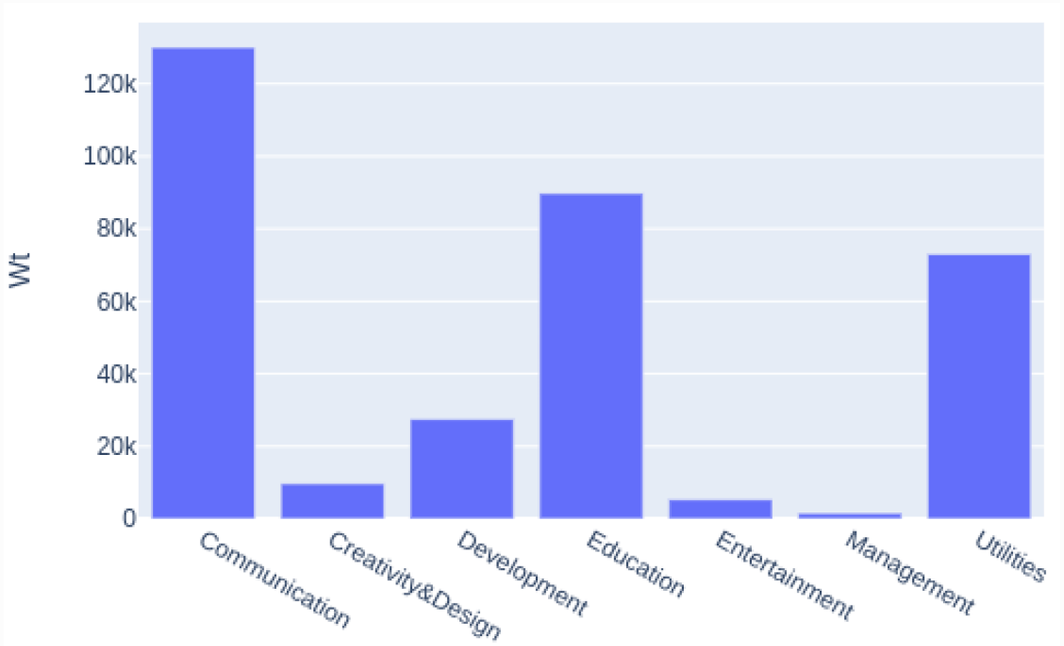
The ICT sector is a major driver of rising energy consumption. By 2030, it could account for 7–20% of global electricity demand. This impact is not only due to hardware (data centers, devices, networks) but also to software. Therefore, we need clear criteria to measure and effectively manage software sustainability.

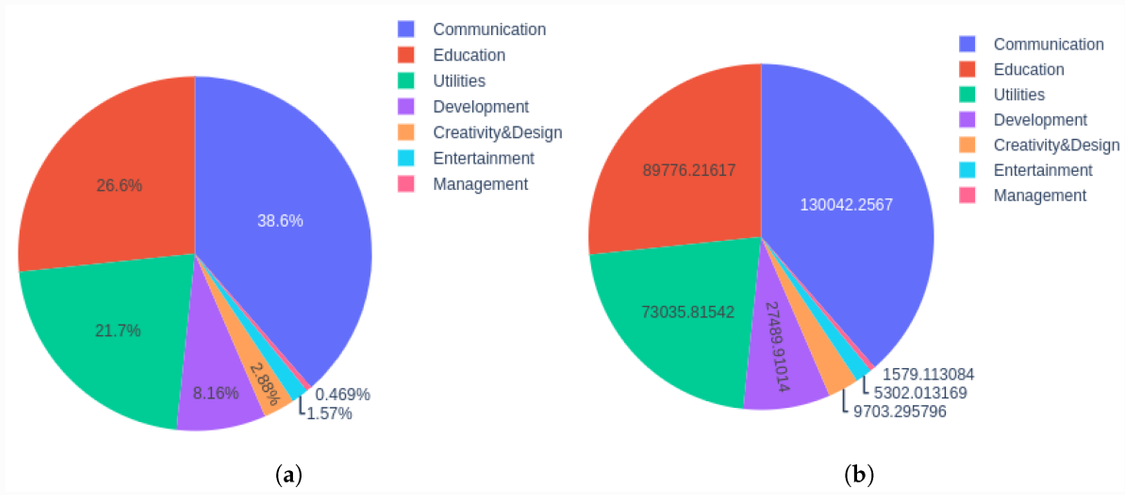


https://www.mullermartini.com/en/newsroom-en/mm-blog/market-trend/digital-is-no-more-sustainable-than-print-%E2%80%93-on-the-contrary!/

Sustainable Software Engineering: ICT and Sustainability

The ICT sector is a major driver of rising energy consumption. By 2030, it could account for 7–20% of global electricity demand. This impact is not only due to hardware (data centers, devices, networks) but also to software. Therefore, we need clear criteria to measure and effectively manage software sustainability.





Ciancarini, P.; Ergasheva, S.; Kholmatova, Z.; Kruglov, A.; Succi, G.; Vasquez, X.; Zuev, E. Analysis of Energy Consumption of Software Development Process Entities. *Electronics* 2020, *9*, 1678. https://doi.org/10.3390/electronics9101678

Sustainable Software Engineering: Sustainability as a Non-Functional Requirement

Sustainability should be treated as a **non-functional requirement (NFR)**, just like performance, security, or usability. It must be considered at **every stage** of the software life cycle, not only at runtime.

Across the Life Cycle

- **Requirements Engineering**: define sustainability goals (e.g., maximum energy budget per feature).
- **Design & Architecture**: select efficient patterns, reusable components, modular design.
- **Implementation**: energy-efficient algorithms, avoid redundant code, optimize data usage.
- **Testing**: include energy profiling and sustainability benchmarks alongside functional tests.
- **Deployment**: choose green hosting, efficient container orchestration, scalable infrastructure.
- **Maintenance**: ensure updates do not increase consumption unnecessarily (e.g., Blue Angel +10% rule).
- **End-of-Life**: support graceful decommissioning, data portability, and long-term compatibility.

Ciancarini, P.; Ergasheva, S.; Kholmatova, Z.; Kruglov, A.; Succi, G.; Vasquez, X.; Zuev, E. Analysis of Energy Consumption of Software Development Process Entities. *Electronics* 2020, *9*, 1678. https://doi.org/10.3390/electronics9101678

Sustainable Software Engineering: Software Sustainability

Software whose development, deployment, and use have **minimal or positive impact** on environment, economy, and society (Dick et al.).

Often referred to as *green software* or *energy-efficient software*.

Key Dimensions:

Environmental: reduces energy use, carbon footprint, resource consumption.

Economic: prolongs hardware lifetime, lowers operational costs, avoids forced upgrades.

Social/Human: ensures usability, accessibility, user autonomy, and avoids manipulative design.

Technical: adaptable, maintainable, modular, and durable over time.

Why It Matters:

Software design choices directly affect energy consumption (e.g., inefficient algorithms, unnecessary background processes).

Poorly designed software accelerates hardware obsolescence (forcing users to buy new devices).

Sustainable software is essential to reach climate and digital responsibility goals.

According to the UN, sustainable development must address the interplay between three dimensions, which are society, the economy, and the environment.

To satisfy the requirements of these dimensions, which

Dick M, Naumann S (2010) Enhancing software engineering processes towards sustainable software product design. In: EnviroInfo. pp 706–715

Sustainable Software Engineering: Software Sustainability

Software whose development, deployment, and use have **minimal or positive impact** on environment, economy, and society (Dick et al.).

Often referred to as *green software* or *energy-efficient software*.

Key Dimensions:

Environmental: reduces energy use, carbon footprint, resource consumption.

Economic: prolongs hardware lifetime, lowers operational costs, avoids forced upgrades.

Social/Human: ensures usability, accessibility, user autonomy, and avoids manipulative design.

Technical: adaptable, maintainable, modular, and durable over time.

Why It Matters:

Software design choices directly affect **energy consumption** (e.g., inefficient algorithms, unnecessary background processes).

Poorly designed software accelerates **hardware obsolescence** (forcing users to buy new devices).

Sustainable software is essential to reach **climate and digital responsibility goals**.

Dick M, Naumann S (2010) Enhancing software engineering processes towards sustainable software product design. In: EnviroInfo. pp 706–715

Sustainable Software Engineering: Sustainability Criteria

Energy efficiency: software should consume the minimum resources necessary
Example: choosing efficient algorithms or avoiding unnecessary background processes.

Compatibility: avoid forcing unnecessary hardware upgrades
Example: applications should still run smoothly on devices that are 5 years old or more.

Usability & accessibility: ensure inclusive and accessible design
Example: support for screen readers, clear interfaces, compliance with WCAG standards.

Durability & modularity: enable reuse, adaptation, and longer lifespan
Example: modular plug-ins instead of monolithic applications, so users install only what they need.

Transparency: provide documented data formats and open APIs
Example: exporting documents in open formats, like CSV, ODF, instead of only proprietary types.

According to the UN, sustainable development needs
to satisfy the requirements of present needs, which
society, the economy, and the environment.

Social impact: respect user autonomy and avoid intrusive advertising
Example: offering opt-in settings, no hidden data collection, and no forced ads.

Sustainable Software Engineering: Sustainability Criteria

Compatibility: avoid forcing unnecessary hardware upgrades
Example: applications should still run smoothly on devices that are 5 years old or more.

Sustainable Software Engineering: Sustainability Criteria

Usability & accessibility: ensure inclusive and accessible design
Example: support for screen readers, clear interfaces, compliance with Web Content Accessibility Guidelines standards.

Sustainable Software Engineering: Sustainability Criteria

Durability & modularity: enable reuse, adaptation, and longer lifespan
Example: modular plug-ins instead of monolithic applications, so users install only what they need.

Sustainable Software Engineering: Sustainability Criteria

Transparency: provide documented data formats and open APIs
Example: exporting files in open formats (e.g., CSV, ODF) instead of only proprietary ones.

Sustainable Software Engineering: Sustainability Criteria

Energy efficiency: software should consume the minimum resources necessary
Example: choosing efficient algorithms or avoiding unnecessary background processes.

Compatibility: avoid forcing unnecessary hardware upgrades
Example: applications should still run smoothly on devices that are 5 years old or more.

Usability & accessibility: ensure inclusive and accessible design
Example: support for screen readers, clear interfaces, compliance with WCAG standards.

Durability & modularity: enable reuse, adaptation, and longer lifespan
Example: modular plug-ins instead of monolithic applications, so users install only what they need.

Transparency: provide documented data formats and open APIs
Example: exporting documents in open formats (e.g., CSV, ODF) instead of only proprietary types.

According to the UN, sustainable development needs

to balance social progress, economic efficiency, and the environment.

Social impact: respect user autonomy and avoid intrusive advertising
Example: offering opt-in settings, no hidden data collection, and no forced ads.

Sustainable Software Engineering: Sustainability Criteria

Energy efficiency: software should consume the minimum resources necessary
Example: choosing efficient algorithms or avoiding unnecessary background processes.

Compatibility: avoid forcing unnecessary hardware upgrades
Example: applications should still run smoothly on devices that are 5 years old or more.

Usability & accessibility: ensure inclusive and accessible design
Example: support for screen readers, clear interfaces, compliance with Web Content Accessibility Guidelines standards.

Durability & modularity: enable reuse, adaptation, and longer lifespan
Example: modular plug-ins instead of monolithic applications, so users install only what they need.

Transparency: provide documented data formats and open APIs
Example: exporting files in open formats (e.g., CSV, ODF) instead of only proprietary ones.

Social impact: respect user autonomy and avoid intrusive advertising
Example: offering opt-in settings, no hidden data collection, and no forced ads.

Sustainable Software Engineering: Measuring Software Sustainability

Hardware-based measurement

Uses external devices (e.g., wattmeters, power analyzers like Janitza UMG 604).

Measures total system power consumption directly.

Advantages: high accuracy, reliable for benchmarks.

Limitations: expensive, complex setup, not always feasible for mobile/IoT devices.

Software-based measurement

Uses estimation tools or APIs (e.g., Microsoft Joulemeter, Intel RAPL, nvidia-smi).

Models resource usage (CPU, GPU, memory, network) to estimate energy consumption.

Advantages: cheaper, easier to apply at scale, works on virtualized/cloud systems.

Limitations: less precise, depends on calibration.

Sustainable Software Engineering: Measuring Software Sustainability

Hardware-based measurement

Uses external devices (e.g., wattmeters, power analyzers like Janitza UMG 604).

Measures total system power consumption directly.

Advantages: high accuracy, reliable for benchmarks.

Limitations: expensive, complex setup, not always feasible for mobile/IoT devices.

Software-based measurement

Uses estimation tools or APIs (e.g., Microsoft Joulemeter, Intel RAPL, nvidia-smi).

Models resource usage (CPU, GPU, memory, network) to estimate energy consumption.

Advantages: cheaper, easier to apply at scale, works on virtualized/cloud systems.

Limitations: less precise, depends on calibration.

Sustainable Software Engineering: Energy-Efficient Programming

Different programming languages have **different energy profiles**:

C / C++ → highly efficient (close to hardware).

Go / Rust → good balance of performance, safety, and efficiency

Java / C# → less efficient due to virtual machines and garbage collection.

Python / Ruby → high-level, but energy-intensive

Studies show C can be **up to 50× more efficient** than Python in some tasks.

Code-Level Decisions

Algorithms: efficient sorting, searching, and ML models reduce energy.

Data structures: choosing linked lists vs arrays can change memory and CPU load.

Concurrency & parallelism: more threads ≠ more efficiency; synchronization overhead can waste energy.

Memory management: frequent garbage collection or memory leaks increase CPU activity → higher consumption.

https://www.green-coding.io/case-studies/energy-efficiency-python/

Sustainable Software Engineering: Energy-Efficient Programming

Different programming languages have **different energy profiles**:

C / C++ → highly efficient (close to hardware).

Go / Rust → good balance of performance, safety, and efficiency

Java / C# → less efficient due to virtual machines and garbage collection.

Python / Ruby → high-level, but energy-intensive

Studies show C can be **up to 50× more efficient** than Python in some tasks.

Code-Level Decisions

Algorithms: efficient sorting, searching, and ML models reduce energy.

Data structures: choosing linked lists vs arrays can change memory and CPU load.

Concurrency & parallelism: more threads ≠ more efficiency;
synchronization overhead can waste energy.

Memory management: frequent garbage collection or memory leaks increase CPU activity → higher consumption.

Sustainable Software Engineering: Blue Angel Framework

The Blue Angel (Blauer Engel), managed by the German Federal Environmental Agency, is the world's first eco-label for sustainable software; although voluntary, it is highly influential in Europe and aims to reduce ICT energy use, extend hardware lifespan, and empower users.

The label is awarded to software that meets strict requirements in three main areas: energy and resource efficiency (ensuring minimal consumption during idle and active use, as well as support for energy-saving functions), hardware longevity (software must remain compatible with devices at least five years old to prevent premature obsolescence), and user autonomy (programs must allow complete uninstallation, modular installation, offline functionality, transparency of data formats and APIs, and must avoid intrusive advertising).

To maintain certification, updates cannot increase energy consumption by more than 10 percent compared to the certified version, and compliance is verified by independent audits.

By incentivizing lightweight, efficient, and user-respecting software, the Blue Angel eco-label plays a pioneering role in promoting digital sustainability and serves as a model for future European and global eco-certifications.

https://www.blauer-engel.de/en

Sustainable Software Engineering: Blue Angel Framework

The Blue Angel (Blauer Engel), managed by the German Federal Environmental Agency, is the world's first eco-label for sustainable software; although voluntary, it is highly influential in Europe and aims to reduce ICT energy use, extend hardware lifespan, and empower users.

The label is awarded to software that meets strict requirements in three main areas: energy and resource efficiency (ensuring minimal consumption during idle and active use, as well as support for energy-saving functions), hardware longevity (software must remain compatible with devices at least five years old to prevent premature obsolescence), and user autonomy (programs must allow complete uninstallation, modular installation, offline functionality, transparency of data formats and APIs, and must avoid intrusive advertising).

To maintain certification, updates cannot increase energy consumption by more than 10 percent compared to the certified version, and compliance is verified by independent audits.

By incentivizing lightweight, efficient, and user-respecting software, the Blue Angel eco-label plays a pioneering role in promoting digital sustainability and serves as a model for future European and global eco-certifications.

https://www.blauer-engel.de/en

Sustainable Software Engineering: Blue Angel Framework

The Blue Angel (Blauer Engel), managed by the German Federal Environmental Agency, is the world's first eco-label for sustainable software; although voluntary, it is highly influential in Europe and aims to reduce ICT energy use, extend hardware lifespan, and empower users.

The label is awarded to software that meets strict requirements in three main areas: energy and resource efficiency (ensuring minimal consumption during idle and active use, as well as support for energy-saving functions), hardware longevity (software must remain compatible with devices at least five years old to prevent premature obsolescence), and user autonomy (programs must allow complete uninstallation, modular installation, offline functionality, transparency of data formats and APIs, and must avoid intrusive advertising).

To maintain certification, updates cannot increase energy consumption by more than 10 percent compared to the certified version, and compliance is verified by independent audits.

By incentivizing lightweight, efficient, and user-respecting software, the Blue Angel eco-label plays a pioneering role in promoting digital sustainability and serves as a model for future European and global eco-certifications.

https://www.blauer-engel.de/en

Sustainable Software Engineering: Blue Angel Framework

The Blue Angel (Blauer Engel), managed by the German Federal Environmental Agency, is the world's first eco-label for sustainable software; although voluntary, it is highly influential in Europe and aims to reduce ICT energy use, extend hardware lifespan, and empower users.

The label is awarded to software that meets strict requirements in three main areas: energy and resource efficiency (ensuring minimal consumption during idle and active use, as well as support for energy-saving functions), hardware longevity (software must remain compatible with devices at least five years old to prevent premature obsolescence), and user autonomy (programs must allow complete uninstallation, modular installation, offline functionality, transparency of data formats and APIs, and must avoid intrusive advertising).

To maintain certification, updates cannot increase energy consumption by more than 10 percent compared to the certified version, and compliance is verified by independent audits.

By incentivizing lightweight, efficient, and user-respecting software, the Blue Angel eco-label plays a pioneering role in promoting digital sustainability and serves as a model for future European and global eco-certifications.

https://www.blauer-engel.de/en

Sustainable Software Engineering: Categories of Sustainability Criteria

Energy & Resource Efficiency

Details

- Software must minimize CPU, GPU, memory, and disk usage.
- Idle consumption should be as close to zero as possible.
- Must support energy-saving features (e.g., suspend, hibernate, efficient caching).
- System requirements should be reasonable and not inflated artificially.

Examples

- **Mozilla Firefox (certified by Blue Angel)** → optimized idle energy use compared to heavier browsers like Chrome.
- **LibreOffice** vs. Microsoft Office → LibreOffice typically has lower idle memory and CPU consumption.
- **Mobile Apps** → apps that use push notifications instead of continuous polling (e.g., Signal Messenger is lighter compared to Facebook Messenger).

Sustainable Software Engineering: Categories of Sustainability Criteria

Hardware Longevity

Details

- Software must run on hardware that is at least **5 years old**.
- Updates cannot significantly increase hardware demand (limit: +10% energy use vs. original certified version).
- Prevents *planned obsolescence* by ensuring older PCs and devices remain usable.

Examples

- **Linux distributions (Ubuntu, Debian)** → often maintain compatibility with old laptops and desktops.
- **Lightweight browsers (Midori, Falkon)** → can run efficiently on older systems where Chrome/Edge struggle.
- **Office suites** like **LibreOffice** → still run on older PCs with modest memory/CPU, unlike some modern cloud-only suites.

Sustainable Software Engineering:

User Autonomy & Transparency

Details

- Full **uninstallation** without leaving hidden background services.
- **Modular installation** → install only needed features (e.g., no forced bloatware).
- **Offline usability** → software should not require constant internet connection.
- **Transparency** → documented data formats, open APIs, no lock-in.
- No hidden advertising, dark patterns, or forced upgrades.

Examples

- **Firefox** → open standards, user control over add-ons, no forced ads.
- **Nextcloud** → fully offline-capable and open-source, transparent data storage.
- **VLC Media Player** → modular codecs, no ads, supports open formats (MP4, MKV, OGG, etc.).
- **Microsoft Teams (negative case)** → difficult to uninstall completely, background processes remain, heavy reliance on online services.

Sustainable Software Engineering: Political and Social Implications

Political Implications

- **Policy & Procurement**: Governments can influence the market by requiring ecological criteria in public procurement of software. For example, procurement guidelines could **mandate Blue Angel–certified software** in schools or public administrations .
- **Regulation & Long-Term Use**: Policies can push software companies to adopt **long-term licensing and update practices**, so hardware does not become obsolete due to software updates .
- **Pricing Models**: Governments may also encourage or regulate **pricing incentives for energy-efficient products**, making sustainable software more competitive .
- **Role of Politicians**: Clear statements and laws are needed to ensure that ecological criteria become **standardized requirements** in software development and use .

Social Implications

- **Education & Awareness**: Integrating sustainability into **computer science curricula** ensures future developers understand how to design energy-efficient and resource-friendly software .
- **Civil Society & NGOs**: Environmental associations can help transfer sustainability knowledge from academia to practice, raising awareness of software's hidden environmental impacts .
- **Consumer Responsibility**: Users can also contribute by choosing sustainable products (e.g., energy-efficient solutions, supporting social projects of lower cost, or ensuring fair labor conditions) .
- **User Empowerment**: Sustainable software enhances user **autonomy**, such as avoiding forced hardware upgrades, enabling uninstallability, and offering offline capability .

Sustainable Software Engineering: Political and Social Implications

Political Implications

- **Policy & Procurement**: Governments can influence the market by requiring ecological criteria in public procurement of software. For example, procurement guidelines could **mandate Blue Angel-certified software** in schools or public administrations .
- **Regulation & Long-Term Use**: Policies can push software companies to adopt **long-term licensing and update practices**, so hardware does not become obsolete due to software updates.
- **Pricing Models**: Governments may also encourage or regulate **pricing incentives for energy-efficient products**, making sustainable software more competitive .
- **Role of Politicians**: Clear statements and laws are needed to ensure that ecological criteria become **standardized requirements** in software development and use.

Social Implications

- **Education & Awareness**: Integrating sustainability into **computer science curricula** ensures future developers understand how to design energy-efficient and resource-friendly software .
- **Civil Society & NGOs**: Environmental associations can help transfer sustainability knowledge from research into society, making the public more aware of software's hidden environmental impacts .
- **Corporate Social Responsibility (CSR)**: Software companies may face growing expectations to adopt **sustainable practices** (e.g., offering energy-efficient solutions, supporting social projects at lower cost, or ensuring fair labor conditions) .
- **User Empowerment**: Sustainable software enhances **user autonomy**, such as avoiding forced hardware upgrades, enabling uninstallability, and offering offline capability .

Sustainable Software Engineering: SoSA - the Software Sustainability Assessment

SoSA is a framework proposed to treat **software systems** as artifacts whose sustainability (beyond just correctness, performance, etc.) should be assessed and designed intentionally. The idea is that software has multiple long-term impacts: energy consumption, maintainability, social consequences, economic viability, etc.

SoSA aims to provide a **multi-dimensional** assessment, making trade-offs explicit, helping decision-makers and architects reason about sustainability over time.

Here are the main elements of SoSA:

Dimensions / Perspectives

SoSA evaluates software along four dimensions (sometimes “pillars”) simultaneously:

1. **Technical** — concerns like maintainability, adaptability, modularity, performance, robustness.
2. **Economic** — cost, return on investment, economic viability, operational cost.
3. **Environmental** — energy consumption, resource use, emissions, waste.
4. **Social** — impacts on users, accessibility, inclusion, privacy, societal effects.

https://research.vu.nl/ws/portalfiles/portal/57104852/SoSA_exercise_report_v05.pdf
https://www.slideshare.net/slideshow/sosa-a-software-sustainability-assessment-method/70369511#5
Patricia Lago. 2019. Architecture design decision maps for software sustainability. In Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '19). IEEE Press, 61-64.

VU VRIJE UNIVERSITEIT AMSTERDAM

Sustainable Software Engineering: SoSA - the Software Sustainability Assessment

Time/Impact Horizon:

In addition to the four dimensions, SoSA distinguishes **three types of impact** over time:

- **Immediate impacts**: those directly observable from the software's use or deployment (e.g. energy used by a query, memory consumption).
- **Enabling impacts**: effects that manifest over time because of the existence or use of the software (e.g. enabling further functionality, increasing resource consumption because more people use it, shortening hardware lifetimes).
- **Systemic impacts**: broader effects at macro / societal level (e.g. behavioral shifts, rebound effects, infrastructure changes).

Thus a design decision might reduce immediate energy usage, but induce a systemic effect of increased usage in aggregate, which SoSA tries to account for explicitly.

VU VRIJE UNIVERSITEIT AMSTERDAM

Sustainable Software Engineering: SoSA - the Software Sustainability Assessment

Modeling & Trade-Offs

- The method encourages building **models** of how software features or architectural decisions propagate through the four dimensions and across the time horizons.
- It helps make **trade-offs transparent**: you can see how a decision that optimizes technical or economic outcomes might worsen environmental or social outcomes, or induce negative systemic effects.
- The framework has been applied (in research) to **Mobility as a Service (MaaS)** systems, modeling from both the **user** and **provider** perspectives.
- In the MaaS case, they used SoSA to see how various features influence environmental, social, technical, and economic outcomes and to compare tradeoffs.

VU VRIJE UNIVERSITEIT AMSTERDAM

Sustainable Software Engineering: SoSA - the Software Sustainability Assessment

Strengths & Benefits

- Encourages **holistic thinking** rather than optimizing one metric at the expense of others.
- Makes **time horizons explicit** (immediate vs long-term/systemic), helping avoid unintended "rebound" or secondary effects.
- Promotes transparency of trade-offs, so stakeholders can see what is being gained and what is being compromised.
- Useful in early design phases: it can guide architectural decisions, feature inclusion, refactoring, etc.
- It has been demonstrated in academic/prototype contexts (e.g. MaaS) showing feasibility of modeling multi-dimensional impacts.

VU VRIJE UNIVERSITEIT AMSTERDAM

Sustainable Software Engineering: SoSA - the Software Sustainability Assessment

Limitations & Challenges

- Complexity: building adequate models is nontrivial. Capturing systemic and social impacts is especially hard.
- Data availability: you might not have good empirical data to parameterize the models (energy usage, social metrics, etc.).
- Trade-off resolution: the framework highlights trade-offs but doesn't automatically give the "right" balance — human judgment and stakeholder input are still needed.
- Scalability: for very large or complex systems, the number of interdependencies can be huge.
- Rebound / second-order effects: even with the framework's awareness, predicting these well is very challenging.
- Lack of standardization or widespread tooling: as of now, SoSA is more a research / conceptual framework than a widely adopted industrial standard.

VU VRIJE UNIVERSITEIT AMSTERDAM

Sustainable Software Engineering: SoSA - Slides

https://assets.new.siemens.com/siemens/assets/api/uuid:32371798-2ab8-44ca-82b0-450900622ce3/Sustainability-at-Siemens.pdf

https://www.slideshare.net/slideshow/sosa-a-software-sustainability-assessment-method/70369511#5

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Criteria for Sustainable Software Products



Gemma Catolino | 2026-2027